SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT: CYBERSECURITY

PRACTICAL - 2

AIM:RSA Encryption and Decryption

Implement the RSA algorithm for public-key encryption and decryption, and explore its properties and security considerations.

CLI:

CODE:

```
from math import gcd

# Function to find modular inverse
def mod_inverse(e, phi):
    for d in range(2, phi):
        if (d * e) % phi == 1:
            return d
    return None

# Input two prime numbers
p = int(input("Enter first prime number (p): "))
q = int(input("Enter second prime number (q): "))

# Calculate n and phi
n = p * q
phi = (p - 1) * (q - 1)

# Choose e
e = int(input("Enter value of e: "))

if gcd(e, phi) != 1:
    print("e must be co-prime with phi(n).")
    exit()

# Calculate d
d = mod_inverse(e, phi)

print("\nPublic Key :", (e, n))
print("Private Key:", (d, n))

# Encrypt
msg = int(input("\nEnter message (integer less than n): "))
cipher = pow(msg, e, n)
print("Encrypted Message:", cipher)

# Decrypt
plain = pow(cipher, d, n)
print("Decrypted Message:", plain)
```

SUMEET YADAV

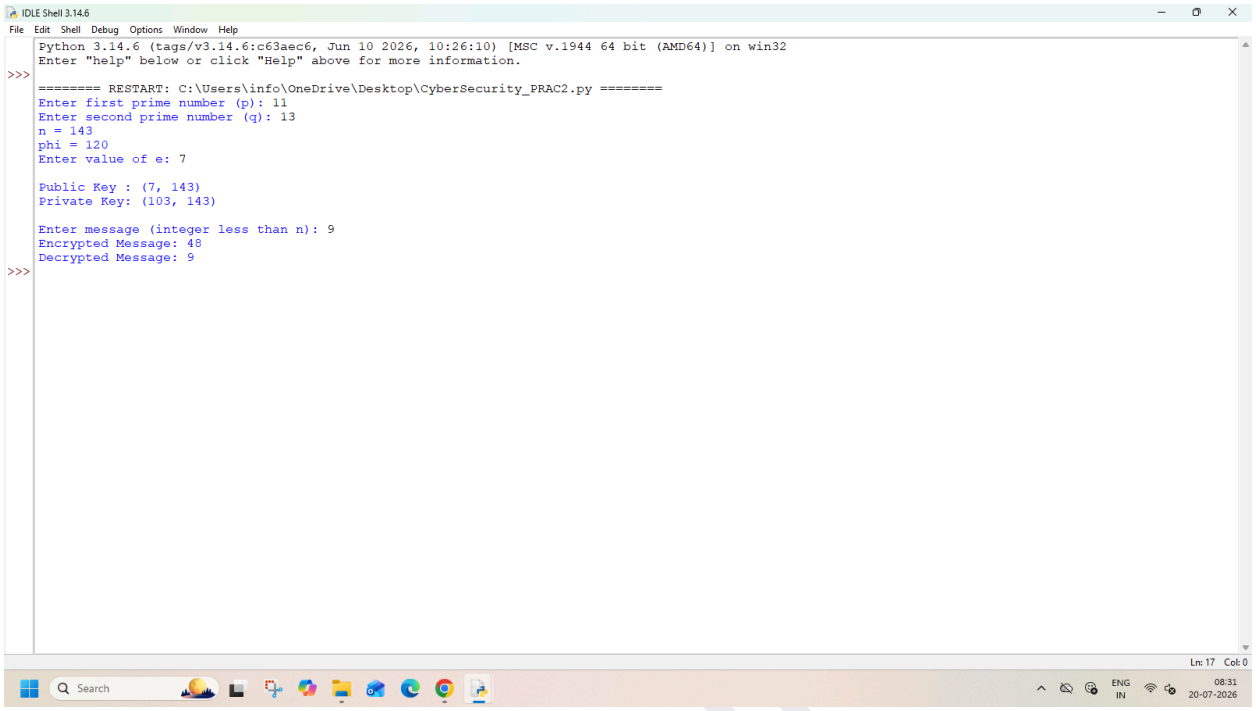
T125

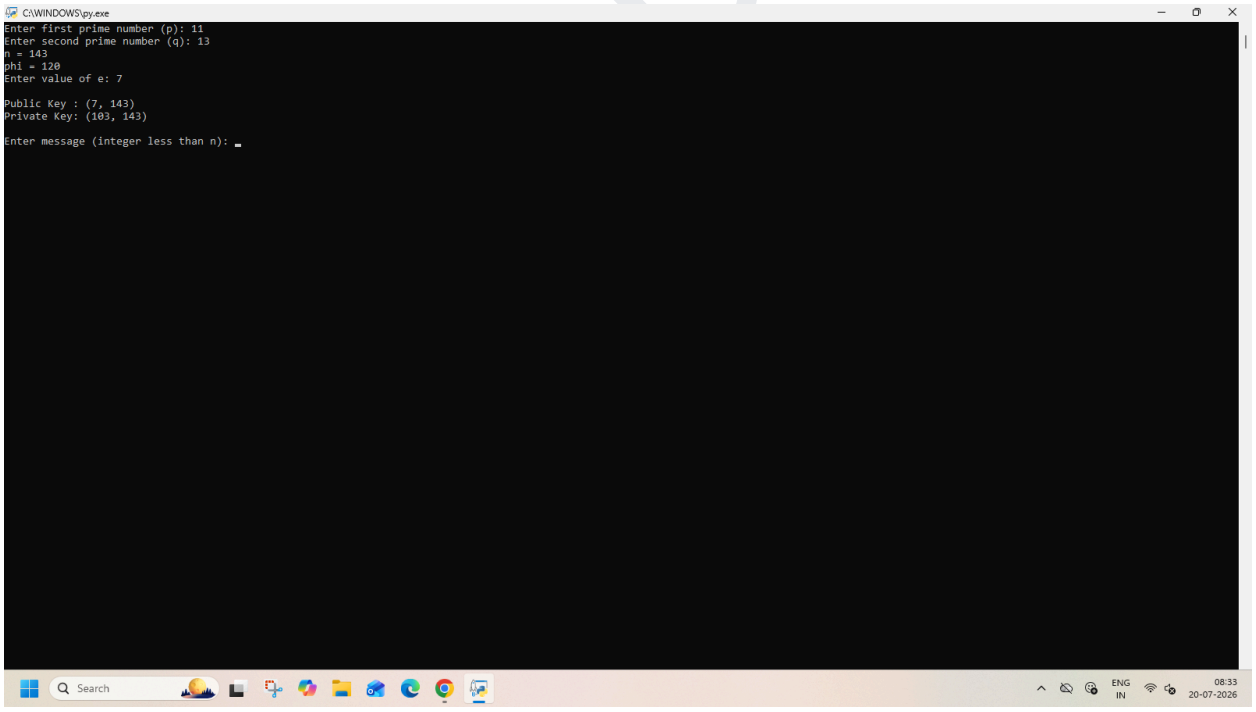
T125_SUMEET

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT: CYBERSECURITY

OUTPUT:





SUMEET YADAV

T125

GUI:

CODE:

```
import tkinter as tk
from tkinter import messagebox
from math import gcd

# Function to find modular inverse
def mod_inverse(e, phi):
    for d in range(2, phi):
        if (d * e) % phi == 1:
            return d
    return None

def generate_keys():
    global n, e, d

    try:
        p = int(entry_p.get())
        q = int(entry_q.get())
        e = int(entry_e.get())

        n = p * q
        phi = (p - 1) * (q - 1)

        if gcd(e, phi) != 1:
            messagebox.showerror("Error", "e must be co-prime with phi(n)")
            return

        d = mod_inverse(e, phi)

        lbl_public.config(text=f"Public Key: ({e}, {n})")
        lbl_private.config(text=f"Private Key: ({d}, {n})")

    except:
        messagebox.showerror("Error", "Enter valid numbers.")

def encrypt():
    try:
        msg = int(entry_msg.get())

        if msg >= n:
            messagebox.showerror("Error", f"Message must be less than {n}")
            return

        cipher = pow(msg, e, n)
```


T125_SUMEET

```
        entry_cipher.delete(0, tk.END)
        entry_cipher.insert(0, str(cipher))

    except:
        messagebox.showerror("Error", "Generate keys first.")

def decrypt():
    try:
        cipher = int(entry_cipher.get())
        plain = pow(cipher, d, n)

        entry_plain.delete(0, tk.END)
        entry_plain.insert(0, str(plain))

    except:
        messagebox.showerror("Error", "Invalid Cipher Text.")

#GUI
root = tk.Tk()
root.title("RSA Encryption & Decryption")
root.geometry("450x400")

tk.Label(root, text="RSA Encryption & Decryption",
         font=("Arial", 16, "bold")).pack(pady=10)

frame = tk.Frame(root)
frame.pack()

tk.Label(frame, text="Prime p").grid(row=0, column=0, padx=5, pady=5)
entry_p = tk.Entry(frame)
entry_p.grid(row=0, column=1)

tk.Label(frame, text="Prime q").grid(row=1, column=0, padx=5, pady=5)
entry_q = tk.Entry(frame)
entry_q.grid(row=1, column=1)

tk.Label(frame, text="Public Key e").grid(row=2, column=0, padx=5, pady=5)
entry_e = tk.Entry(frame)
entry_e.grid(row=2, column=1)

tk.Button(root, text="Generate Keys",
          command=generate_keys,
          bg="lightblue").pack(pady=10)

lbl_public = tk.Label(root, text="Public Key:")
lbl_public.pack()
```


T125 - SUMEET

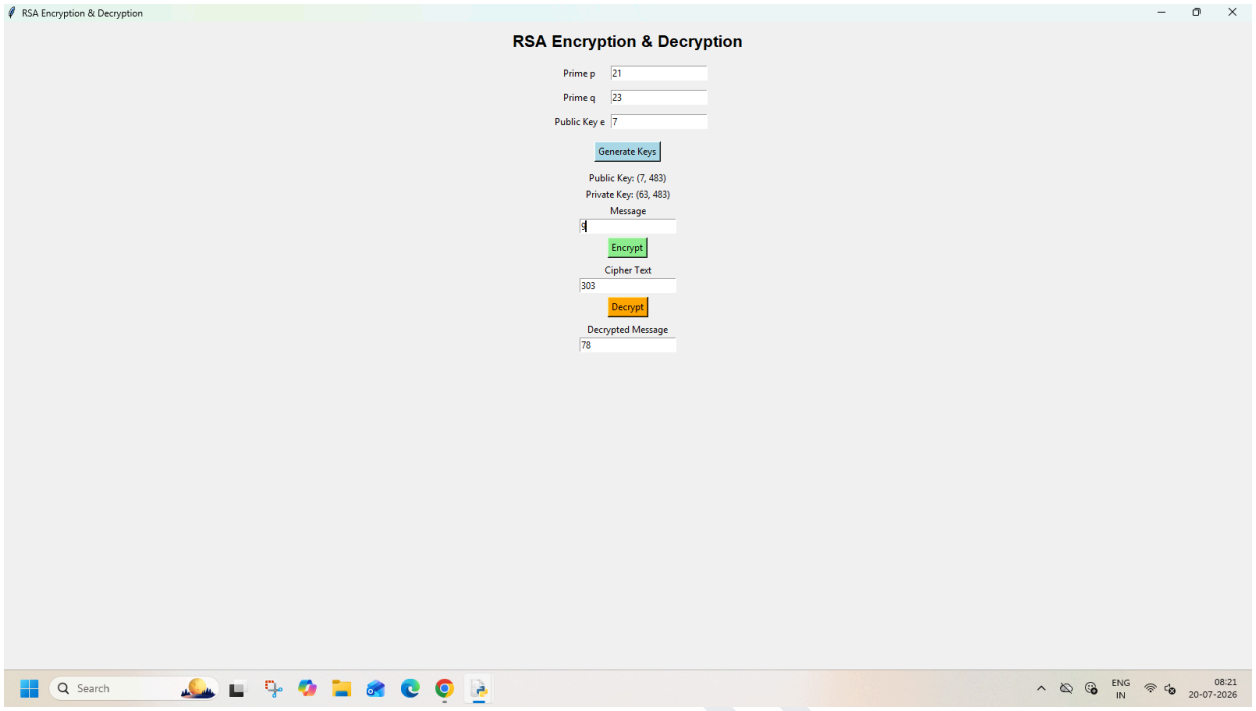
```
lbl_private = tk.Label(root, text="Private Key:")
lbl_private.pack()

tk.Label(root, text="Message").pack()
entry_msg = tk.Entry(root)
entry_msg.pack()

tk.Button(root, text="Encrypt",
        command=encrypt,
        bg="lightgreen").pack(pady=5)

tk.Label(root, text="Cipher Text").pack()
entry_cipher = tk.Entry(root)
entry_cipher.pack()

tk.Button(root, text="Decrypt",
        command=decrypt,
        bg="orange").pack(pady=5)

tk.Label(root, text="Decrypted Message").pack()
entry_plain = tk.Entry(root)
entry_plain.pack()

root.mainloop()
```

T125 _ SUMEET

SHETH L.U.J AND SIR M.V. COLLEGE

SUBJECT: CYBERSECURITY

OUTPUT:



T125 - SU

SUMEET YADAV

T125